Coding Like a Duck

If you want to write your programs using the duck typing philosophy, you really need to remember only one thing: an object's type is determined by what it can do, not by its class.

What does this mean in practice? At one level, it simply means that there's often little value explicitly checking for the class of an object.

For example, you may be writing a routine to add song information to a string. If you come from a C# or Java background, you may be tempted to write this:

```
def append_song(result, song)
  unless result.kind_of?(String)
    fail TypeError.new("String expected")
  end
  unless song.kind_of?(Song)
    fail TypeError.new("Song expected")
  end

  result << song.title << " (" << song.artist << ")"
end

result = ""
append_song(result, song)
```

Embrace Ruby's duck typing, and you'd write something far simpler:

```
def append_song(result, song)
  result << song.title << " (" << song.artist << ")"
end

result = ""
append_song(result, song)
```

You don't need to check the type of the arguments. If they support << (in the case of result) or title and artist (in the case of song), everything will just work. If they don't, your method will throw an exception anyway (just as it would have done if you'd checked the types). But without the check, your method is suddenly a lot more flexible. You could pass it an array, a string, a file, or any other object that appends using <<, and it would just work.

Now sometimes you may want more than this style of *laissez-faire* programming. You may have good reasons to check that a parameter can do what you need. Will you get thrown out of the duck typing club if you check the parameter against a class? No, you won't.The duck typing club doesn't check to see whether you're a member anyway…. But you may want to consider checking based on the object's capabilities, rather than its class:

```
def append_song(result, song)
  unless result.respond_to?(:<<)
    raise TypeError.new("'result' needs `<<' capability")
  end
  unless song.respond_to?(:artist) && song.respond_to?(:title)
    raise TypeError.new("'song' needs 'artist' and 'title'")
  end
  result << song.title << " (" << song.artist << ")"
end

result = ""
append_song(result, song)
```